Resource Management Schemes for Cloud Native Platforms with Computing Containers of Docker and Kubernetes

Ying Mao, *Member, IEEE,* Yuqi Fu Suwen Gu, *Student Member, IEEE,* Wenrui Mu
Long Cheng, Qingzhi Liu, *Member, IEEE*

arXiv:2010.10350v2 [cs.DC] 29 Sep 2025

***Abstract*—Businesses have made increasing adoption and incorporation of cloud technology into internal processes in the last decade. The cloud-based deployment provides on-demand availability without active management. More recently, the concept of cloud-native application has been proposed and represents an invaluable step toward helping organizations develop software faster and update it more frequently to achieve dramatic business outcomes. Cloud-native is an approach to build and run applications that exploit the cloud computing delivery model's advantages. It is more about how applications are created and deployed than where. The container-based virtualization technology, e.g., Docker and Kubernetes, serves as the foundation for cloud-native applications. This paper investigates the performance of two popular computational-intensive applications, big data and deep learning, in a cloud-native environment. We analyze the system overhead and resource usage for these applications. Through extensive experiments, we show that the completion time reduces by up to 79.4% by changing the default setting and increases by up to 96.7% due to different resource management schemes on two platforms. Additionally, the resource release is delayed by up to 116.7% across different systems. Our work can guide developers, administrators, and researchers to better design and deploy their applications by selecting and configuring a hosting platform.**

***Index Terms*—Container Management; Docker Swarm; Kubernetes; Tensorflow; Pytorch; Apache Yarn and Spark;**

I. Introduction

In the past decade, businesses from all over the world, regardless of size or industry, have been turning to cloud services to fulfill their computing needs. Among the benefits of embracing cloud services at a corporate level are that moving to cloud computing can reduce the cost of managing and maintaining the IT systems and those cloud infrastructures can be scaled up or down based on the computation and storage needs quickly to suit the business development, allowing flexibility as the needs change.

According to a Forbes report [1], spending on cloud computing infrastructure and platforms grew at a 30% compound annual growth rate from 2013 through 2018 compared with 5% percent growth for overall enterprise IT. From the cloud computing services providers perspective, such as Amazon Web Service [2], Microsoft Azure [3] and Google Cloud Platform [4], virtual machine backed virtualization technology powers the datacenters and provides services to multiple users on the same server while maintains isolation among them. From the client's perspective, firstly, they develop and test applications locally and then, create a virtual machine on the cloud, which is identical to their run-time environment, e.g., operating system and hardware settings, and finally, deploy the applications inside the virtual machines to provide the services.

In a small-scale system, the system is efficient since clients can purchase to replicate the active virtual machines when their demands are increasing and terminate the virtual machines when they need to scale down. As the system grows, however, providing services through virtual machines implies running many duplicated instances of the same OS and redundant boot volumes, which lead to large scheduling overhead. Additionally, virtual machines usually target providing full services, e.g. access control, database, and logging on one type of machine. Although replicating virtual machines can improve the performance of database services when the workload of database queries is increasing, the whole system will inevitably waste some of the resources that are assigned for other services on the same virtual machine.

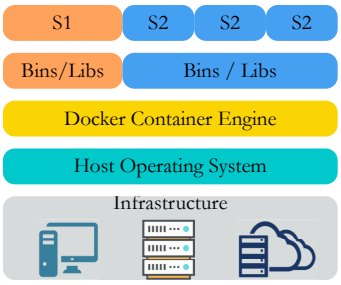


Figure 1: Docker Containers

To address the above limitations and fully utilize the advantages of cloud computing, containerization tools, such as Docker [5], are designed for deploying and running distributed applications without launching entire virtual machines. Instead, multiple isolated containers share the host operating system and physical resources. Furthermore, containerization is service level virtualization in that each container focuses on providing a limited number of services, which makes the system more flexible for the elasticity of demands.

Fig. 1 shows a machine that hosts four containers, which target on providing two services (S1 and S2). Instead of loading the whole guest operating system, Docker only installs the required binaries and libraries, which saves significant amounts of resources and greatly speeds up the booting process. Currently, S1 is offered by one container, while S2 is offered by two. If S1's workload increases, the system can initiate a new container for S1 or, depending on S2's situation, it may stop one of S2's containers and direct the resource towards to S1.

Y. Mao, Y. Fu, S. Gu, and W. Mu are with the Department of Computer and Information Science at Fordham University in the New York City. E-mail: {ymao41, yfu81, sgu21, svhaduri}@fordham.edu

L. Cheng is with the School of Computing, Dublin City University, Ireland. E-mail: long.cheng@dcu.ie

Q. Liu is with Information Technology Group, Wageningen Univer- sity, The Netherlands E-mail: qingzhi.liu@wur.nl

With container-based virtualization, the concept of cloud-native is proposed to develop applications built as services that are packaged in virtualized containers, deployed as microservices and managed on elastic infrastructure through agile development, operation processes and continuous delivery workflows. In a production environment of cloud-native applications, a multi-node cluster is used to provide the backend infrastructures for running at scale. In the domain of managing clusters for containerization, Docker Swarm [6] and Kubernetes [7] are the key players. Both of them provide rich functionalities for resource management schemes, security policy enforcement, network access, system administration, etc. However, these functionalities would introduce overhead to the system. In this paper, we investigate the performance of using cloud-native frameworks from the perspective of resource management. The main contributions of this paper are summarized as follows,

- We build a monitor system to collect real-time resource usage data as well as calculating completion time on Docker and Kubernetes platforms for containerized big data and deep learning applications.
- We conduct intensive experiments on each individual platform with different workloads and discover that the completion time can be reduced for up to 79.4% and 69.4% by changing the default configurations on Docker and Kubernetes, respectively.
- Through extensive experiments, we demonstrate that the Docker platform delayed the resource release process for up to 116.7% for short-lived deep learning jobs. Furthermore, the completion time can increase up to 96.7% when compared with each other due to different resource management schemes.

The remainder of this paper is organized as follows. In Section II, we review the related works in the literature. In Section III, we introduce the cloud-native applications as well as two representative hosting platforms, Docker and Kubernetes, along with their resource management schemes. Section IV presents the comprehensive experiments that we conducted to evaluate the platforms and the analytical results that generated by the applications. Finally, Section V concludes the paper.

II. Related Work

Cloud computing has been commercialized and started serving the industry over a decade [8]. Whenever we scroll through Facebook [9] or Twitter [10], watch streaming videos, access an online email service like Gmail [11] or Outlook [12], or use the applications on our phones, we are accessing data and services on the cloud. Based on cloud infrastructures, many research projects have been conducted to optimize system performance [13]–[16] and enable new applications [17]–[19]. Cloud computing is, traditionally, powered by virtual machines in order to share the resources among different clients and maintain an isolated environment for each of them. While this architecture works great for a small scale system, it suffers from large overhead on service provision, storage spaces, and system elasticity [20] when serving a larger and more complicated system.

More recently, the concept of cloud-native computing is proposed for architecting applications specifically to run in the elastic and distributed nature, which required by modern cloud computing platforms. At the back end side, container-based virtualization technologies serve as the foundation of cloud-native applications. Comparing with virtual machines, containers are more lightweight, resource-friendly, and efficient to scale up [21]. There are lots of works investigate the performance of containers and virtual machines for various perspectives. A comparative study of different virtualization technologies is conducted in [22], the authors conclude that while containers may offer bare-metal performance, they are lack of rich isolation properties. ContainerCloudSim [23] extends CloudSim for modeling and simulation of containerized cloud computing environments. The authors in [24] present a mathematical framework for decision making around placing and migrating workloads in a data-center where applications are packaged as OS containers running on virtual machines. In the domain of big data processing, experiments in [25], [26] demonstrate that containers enhance the performance of enterprise clouds.

Besides benefits offered by individual containers, a more advanced part of cloud-native computing is container orchestration. In this domain, Docker Swarm [6] and Kubernetes [7] are the key players. Asif et. al provides a non-exhaustive and prescriptive guide in [27] to identifying and implementing key mechanisms required in a container orchestration platform. Eddy et. al study the e performance overhead of container orchestration frameworks for running and managing database clusters. NBWGuard [28] is proposed as a design for network bandwidth management for Kubernetes. With emerging containerized deep learning applications, authors in [29] propose ProCon, a start-of-art container scheduler that takes the progress of each job into resource allocation. Furthermore, in order to optimize the container management platform for the model training process, TRADL [30] and FlowCon [31] provide efficient scheduling algorithms to boost the scalability of the systems.

In addition to the existing research, in this paper, we investigate the resource usage patterns of cloud-native applications in two popular types, deep learning framework (e.g. Tensorflow [32] and Pytorch [33]) and big data processing [34].

III. Applications on the cloud

In this section, we will discuss cloud-native applications and container technologies, such as Docker and Kubernetes, which are the fundamental building block of cloud-native applications.

A. Cloud Native

Cloud-native is generally an approach to build and run applications that leverage the benefits of the cloud computing model. The Cloud Native Computing Foundation (CNCF) [35] characterizes cloud-native as using open source software stack with the following properties.

- Containerized, which means each part of the application is packaged into its own container. A property that enables reproducibility, transparency, and resource isolation.

- Dynamically orchestrated. Containers are actively scheduled and regulated to optimize resource utilization.
- Microservices-oriented. Applications are divided into microservices and managed on elastic infrastructure to improve the overall agility and maintainability of applications.

A container is a standard unit of software that packages up the programs and all their dependencies together. Containers isolate an application from its environment and guarantee that it works reliably across different environments. Cloud-native applications, on the other hand, are a collection of independent services that are packaged as lightweight containers.

Different from virtual machines, containers virtualize operating systems instead of hardware, which makes them more portable, efficient, and scalable. Containers are an abstraction at the application layers and multiple containers can run on the same machine while each running as an isolated process. The images of containers have a layered structure. Fig. 2 presents a sample image for running a container of Apache Spark. We first pull the Ubuntu Operating System (L1) as the base from DockerHub [36]. On top of the OS layer, we load the maven (L2) for Java-related packages and then, Hadoop HDFS (L3) is built on top of L2. Finally, we fetch Spark (L4) for data processing. Compare to virtual machines, containers occupy less space, can handle more applications with fewer hardware resources.

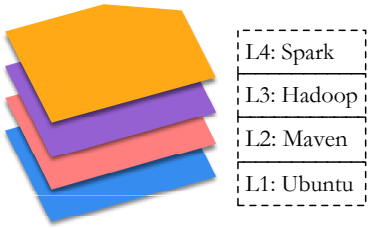


Figure 2: Sample Container Images

B. Container Orchestration

Cloud-native applications are developed as a system of multiple, relatively small microservices that work together to provide a comprehensive system. Services could be developed and deployed independently where each realizes exactly one functionality so when change is required in a certain part of the application, only the related services need to be modified and redeployed. Besides, the services of a cloud-native application could use different types of languages, runtimes, and frameworks. Each service is could beeveloped using the language and framework that best suited for specific functionality.

Manual application deployment and management could be challenging, because monitoring the system status, managing active containers, and balancing the load between running applications at the same time is costly and respond slowly to unexpected changes in the system. Cloud-native applications use automatically container orchestration tools such as Docker Swarm and Kubernetes to be fully benefited from a cloud platform.

1) Docker Swarm Mode: While Docker creates virtualized containers on single machines, Docker Swarm Mode is an open-source container orchestration platform, which provides a suite of management toolkit for a cluster of containers. It is the native clustering solution for Docker containers, which has the advantage of being firmly integrated into the Docker ecosystem and utilizing its own API. A swarm consists of multiple Docker hosts that run in swarm mode.

Fig. 3 illustrates the system architecture of Docker swarm mode in a multi-node cluster. There are two types of nodes in this cluster, worker nodes, and manager nodes. Worker nodes are responsible for running tasks; on the other hand, manager nodes accept specifications from the user and pull images from sources (e.g. Docker Hub). Worker nodes receive and execute tasks from managers. The agent on the worker node reports the state of tasks to the manager, who is responsible for reconciling the desired state with the actual cluster state. Services define how individual containers can distribute themselves across the nodes. The Task is a container and the commands to be run inside of the container. Each manager is in charge of one or multiple workers. The number of managers can increase along with the workload.

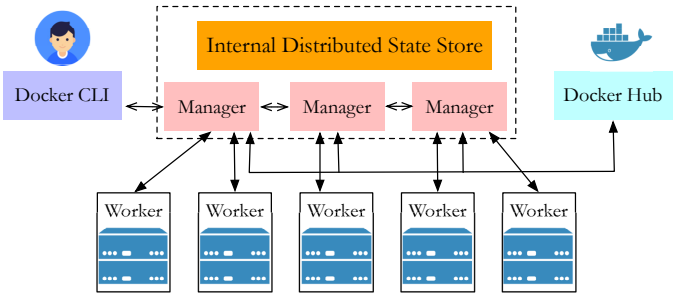


Figure 3: Docker Swarm Mode

2) kubernetes: Kubernetes is an open-source platform created by Google for container deployment operations, scaling, and management of a containerized application. It is a management tool that has been specifically designed to simplify the scalability of workload using containers. Kubernetes supports the same images of Docker.

Fig 4 plots the architecture of a Kubernetes cluster. Similar to Docker Swarm, the system has managers and workers. The clients can use kubectl, a command-line interface for running commands against Kubernetes clusters, to interact with API server, which resides in the manager. The manager node is responsible for the Kubernetes cluster and takes care of orchestrating the worker nodes by using the controller and scheduler. It utilizes etcd for the services of consistent and highly-available key-value stores. On the worker side, kebulet is running on each node in the cluster. It maintains the status of containers on this worker. A group of containers, one or multiple, on the same node that is created, scheduled, and deployed together is called a pod. This group of containers would share storage, namespace, cgroups, IP addresses. Kubernetes also introduces the concept of a service, which is an

abstraction on top of a number of pods, typically requiring to run a proxy, for other services to communicate with it via virtual IP addresses. Once the system is online, workers communicate with managers for configurations of containers downloading the images and starting the containers.

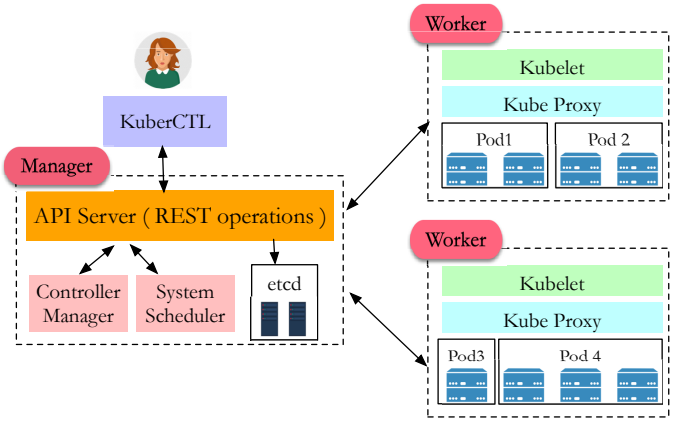


Figure 4: Kubernetes

C. Resource Management for Cloud Native Applications

In a busy cloud computing platform, the resource contention happens all the time. From the operating system's point of view, the active containers are not different from regular processes, where the resource allocation is controlled by the system. Considering a cloud-native application, which powered by a cluster of containers, two levels of resource management schemes exist: individual worker level and container placement level.

1) Individual Workers: Different from fixed resource configurations for virtual machines, containers employ various plans for resource management.

Docker Platform: When initiating, no resource constraints, by default, are applied to newly created containers. In this scenario, each one can use as much of a given resource as the host's kernel scheduler allows. Multiple containers compete with resources freely in the system. When necessary, developers have the option to start one with specific resource limits. For example, the command, `docker run -it --cpus=\.7" spark /bin/bash`, guarantees the new container at most 70% of the CPU (one core) every second. Please note that Docker utilizes the "soft limit" concept, where the value serves as the upper limit to containers and if the actual usage can not top-up the limits, the reminding resources could be used by others. In addition to preset constraints, users can update the configuration at runtime. For instance, the command, `docker run -dit --name spark --memory 2048M`, dynamically limit the memory usage of a "spark" container to 2G. The supported resource types are CPU, memory, disk I/O, and network I/O.

Kubernetes Platform: Similar to Docker, when clients specify a Pod, they can optionally specify how much CPU and memory each container needs, in terms of limits and requests. A limit is the maximum amount of resources that Kubernetes will allow the container to use.

```
resources:
  limits:
    memory: "200Mi"
  requests:
    memory: "100Mi"
```

Figure 5: YAML

A request is the amount of that resources that the system will guarantee for the container, and Kubernetes will use this value to decide on which node to place the pod. For example, Fig. 5 shows a piece of YAML [37] file that the developer specified for memory spaces.

Kubernetes employs three resource assignment policies, which aim to provide different Quality of Service (QoS) classes.

- Guaranteed: Every Container in the Pod must have memory and CPU limit as well as a memory and CPU request, and they must be the same.
- Best Effort: If a Pod does not have any memory or CPU limits or requests, it is in the "best-effort" mode, where it competes for resources freely with others.
- Burstable: The Pod does not meet the criteria for Guaranteed and at least one container in the Pod has a memory or CPU request.

2) Container placement: In a cluster, typically, there are multiple workers serve as computing nodes. For an incoming job request, the manager has to decide which worker should host the task. Because workers' status, such as the number of active jobs and resource availabilities varies, different container placement strategies lead to various performance.

Docker Swarm: There are three container placement schemes. (1) Spread (default) aims to place a container on the node with the fewest running containers; (2) Binpack places a container onto the most packed node in the cluster; (3) Random, randomly pick up a worker as the host.

Kubernetes: Usually, Kubernetes scheduler automatically determines an appropriate node for the pod with a scoring algorithm, which calculates a score to every worker based on multiple factors such as available resources. However, there are situations when more control is needed for the placement. A pod can request a node with specific characteristics through node affinity and it is also possible to ensure that a set of pods are placed on the same node to avoid latency. Placement strategies can be implemented by a set of labels and annotations appropriately included or excluded within pods and nodes.

IV. Performance Evaluation

In this section, we explore two popular and representative container platforms, Docker Swarm and Kubernetes.

A. Experiment Setup, Workloads and Evaluation Metrics

1) Monitoring the resources: We build a container monitor system based on Prometheus [38] and Grafana [39]. Fig. 6 shows the system that we used for our experiments. In this system, Prometheus daemon is running on each worker to collect the resource usage data of all the containers/pods on this node. The data is collected by the managers and analyzed, presented by Grafana.

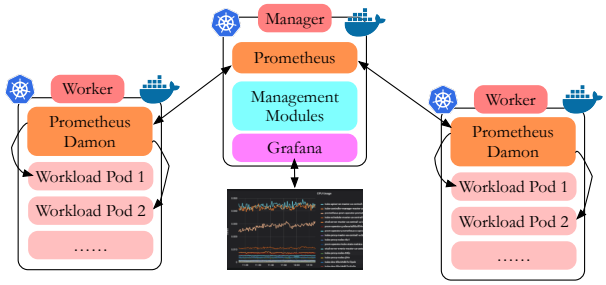


Figure 6: Resource Monitoring

2) Workloads and Testbeds: In the experiments, we mainly evaluate two types of applications, big data (BD) processing jobs, and deep learning (DL) training jobs. For big data processing, we evaluate Apache Yarn [40] and Apache Spark [34], which are popular distributed general-purpose big data processing framework. For the deep learning applications, we focus on two frameworks, Tensorflow [32] and Pytorch [33] that are widely adopted for training deep learning models in the industry.

Table I presents the experiments that we conduct for evaluating the container orchestration toolkits.

Table I: Workload for Experiments

Application	Platform	Category
Variational Autoencoders	Tensorflow	DL
MNIST - CNN	Tensorflow	DL
MNIST - RNN	Tensorflow	DL
MNIST - Identity MRNN	Pytorch	DL
VAE - MLP	Pytorch	DL
PageRank	Spark	BD
WordCount	Yarn	BD

We run experiments on both bare-metal machines and small-scale clusters. In addition, the evaluation is conducted on NSF Cloudlab [41], which is hosted at the University of Utah, Downtown Data Center. Specifically, we use M510 as our physical machines for both standalone and cluster experiments. It contains 8-core Intel Xeon D-1548 at 2.0 GHz, 64GB ECC Memory and 256 GB NVMe flash storage.

To evaluate the platforms from different perspectives, we conduct the following types of experiments.

- Type-1: 1 job in 1 container/pod (1 in 1): this set of experiments focuses on how the resource is allocated for limited workload within a container.
- Type-2: 5 jobs in 1 container/pod (5 in 1): this set of experiments investigates the resource allocation for the same container with an increased workload.
- Type-3: 5 jobs in 5 containers/pods (5 in 5): this set of experiments evaluate the system performance with a slightly larger number of individual containers.
- Type-4: 5 jobs in 5 container/pods with 100s submission interval in Deep learning jobs and 20s interval for Hibench big data jobs: this type of experiment introduces overlaps between computing jobs, which leads to more scheduling tasks for the system.
- Type-5: Mix number of different jobs with a random submission schedule: this type of experiment requires more scheduling tasks to be performed by the system.

When evaluating the systems, we conduct an experiment with and without resource limits. Whenever the last job in the system is done, we terminate the containers in the system. Throughout the experiments, we focus on the two key metrics to assess the platforms, completion time, and resource usage.

- Completion Time: the completion time of each individual job. Note that each container runs one or multiple jobs and each pod consists of one or multiple containers.
- Resource Usage: resource usage when achieving the corresponding completion time per container/pod level.

Additionally, we conduct experiments with the following job submission schedules.

- Burst schedule: it simulates a server with simultaneous workloads, which is a challenge for the system to adjust resources according to each individual job.
- Fixed schedule: the time to launch a job is controlled by the administrator. When a new job joins the system, the system will have to redistribute the resources to accommodate it.
- Random schedule: the launch times are randomized to simulate random submissions of jobs by users in a real cluster. The system has to frequently adjust resources to achieve its designed objectives.

B. Evaluation Results

The evaluation result is organized into three parts in terms of different job categories, deep learning jobs, big data processing jobs, and a cluster with mixed settings.

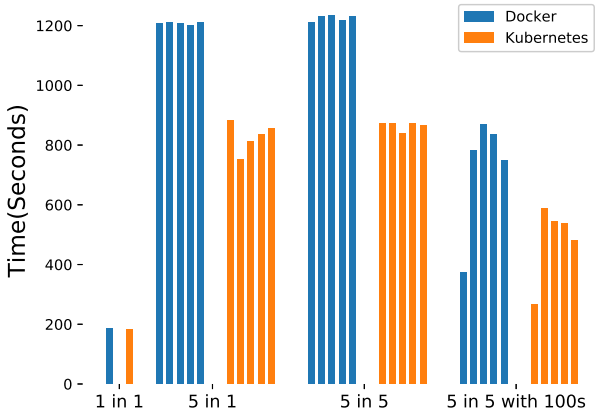
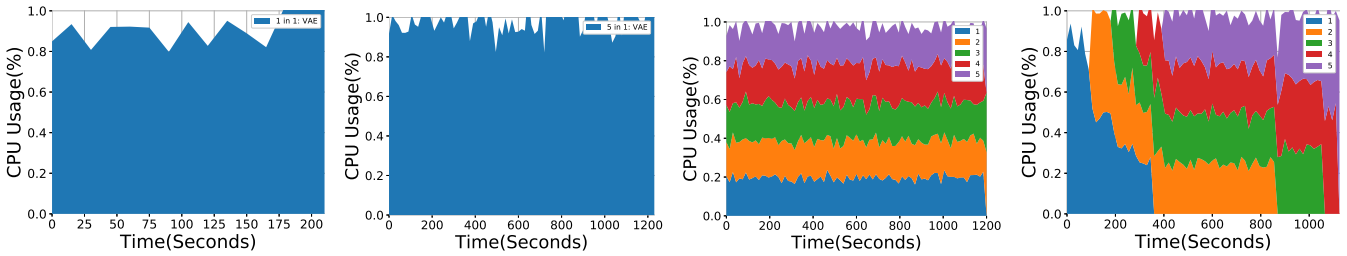


Figure 7: Completion time of deep learning jobs

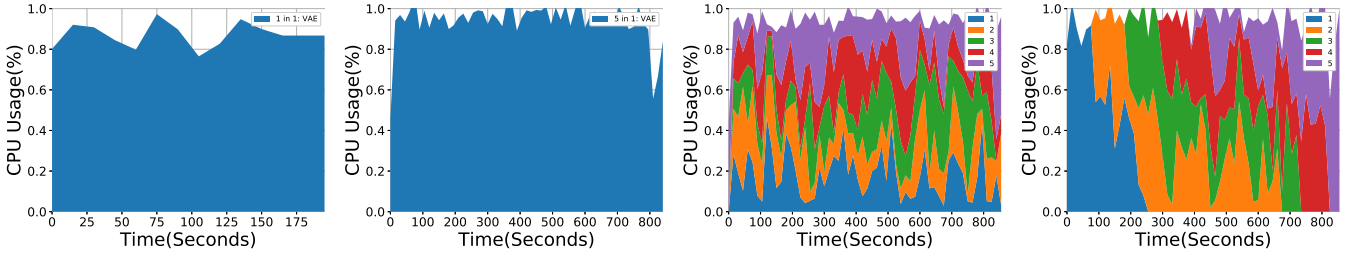
1) Deep Learning Jobs: Firstly, we conduct the above-mentioned Type 1 to 4 with deep learning jobs. We use Variational Autoencoders (VAE) on Tensorflow as a representative workload. We utilize the default Tensorflow configuration for the experiments. Since deep learning applications are computation-intensive, we focus on the CPU usage in this subsection.

Completion Time: Figure 7 plots the completion time of each type of experiment. From the figure, we can see that if



(a) 1 in 1 VAE (b) 5 in 1 VAE (c) 5 in 5 VAE (d) 5 in 5 VAE with 100s interval

Figure 8: CPU usage for VAE with Docker Platform



(a) 1 in 1 VAE (b) 5 in 1 VAE (c) 5 in 5 VAE (d) 5 in 5 VAE with 100s interval

Figure 9: CPU usage for VAE with Kubernetes Platform

there is only 1 job running in the system, both Docker and Kubernetes achieve similar results (186.1s v.s. 183.3s). When the number of containers/pods increases, the performance of Docker and Kubernetes varies a lot. Comparing the platforms with Type 2 experiments, where 5 VAE training jobs are running in 1 container, Kubernetes records a 37.7% reduction on Job-2, which finishes at 753.2s and the same job of Docker completes at 1209.6s. The average completion time of 5 jobs is 1207.1s and 815.8s, an average 32.5% reduction among 5 jobs. Additionally, the standard deviation for the experiments is 4.47 and 38.79 for Docker and Kubernetes, respectively. It indicates that, for the same amount of resources, Kubernetes achieve far better performance with intensive resource competition.

A similar trend is found with Type 3 experiment such that 5 VAE models are running concurrently in 5 containers. For example, the largest gain is discovered on Job-3, which reduces from 1233.7s to 840.8s, 31.87%. On average, the improvement of completion time is 29.4%, from 1223.8s to 863.6s. The standard derivations for Docker and Kubernetes are 8.12 and 12.02, respectively.

The same trend is observed again in the Type 4 experiments that 5 VAE jobs that run in their own container and are submitted to the system with 100s interval. The average completion time reduces from 772.2s to 482.6s for Docker and Kubernetes. The largest gain is found at Job-3, who submits at 200s in the experiment and records 37.4% reduction. The standard derivations are 178.84 (Docker) and 113.49 (Kubernetes). The values are significantly larger than all previous experiments since jobs are submitted into the system with an interval of 100s, which means Job-1 has 100s to occupy the entire worker and Job-2 has 100s to share resources with only another one. The uneven resource distribution leads to different completion times.

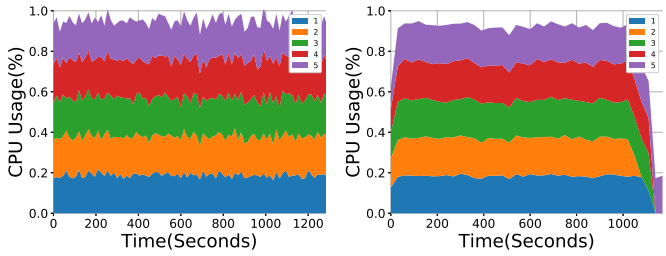
When we compare experiment Type 3 with Type 2 experiments on the same platform, the completion time for both Docker and Kubernetes increases. For example, the completion time of Job-2 in Docker and Kubernetes increases from 1209s to 1232s and 753s to 871s. On average, it increases from 1207.1s to 1233.7s (Docker) and 815.8s to 863.6s. This is because, with the same computing platform Docker or Kubernetes, when 5 jobs running in the 5 individual containers/pods, the overhead is larger than the same workload running in one single container/pod. More containers/pods lead to more repetitive works.

Comparing to the two platforms, the system overhead can result in completion time variations, however, it fails to explain the huge gaps in performance.

Resource Usages: Next, we dig into the details of the previous experiments. Figure 8 and Figure 9 illustrate the CPU usage at the container (Docker) and pod (Kubernetes) level for the two platforms.

When only 1 job is running in the system, Figure 8a and Figure 9a plot the data for Docker and Kubernetes, respectively. We can see that the container in Docker uses a bit more CPU resources, 87% on average than 83% for Kubernetes. This is due to the fact that Kubernetes utilizes an additional layer, pod, to manage a group of containers and thus, generate slightly more overhead with this 1 job workload. With the same training job, however, Kubernetes is stopped at 195s and Docker terminates at 212s. According to the completion time data, the job that runs inside each platform is 183.3s and 186.1s for Kubernetes and Docker. Therefore, Docker requires more time to terminate a running container and completely release resources (26s vs 12s).

Next, we create a container/pod and submit 5 VAE jobs to the system. Figure 8b and Figure 9b present the CPU usage.



(a) Docker (b) Kubernetes

Figure 10: Experiments of 5 in 5 with specific limits

We can find a similar trend that the container ins Docker consumes almost all the CPU usages and Kubernetes still reserves a small number of resources to the system itself.

Considering completion time, while for Type 1 tests, both platforms achieve similar results, there is a huge performance gap when comparing them in Type 2, 3, 4 experiments, which fails to explain by system overhead.

Taking Fig. 8c and Fig. 9c as an example, we find that, with the same workload and configuration, Docker and Kubernetes have very different resource management schemes. In Fig. 8c (Docker), each container is allocated an equal share of the resources, around 20%, and maintains the same amount throughout the whole runtime. With Kubernetes (Fig. 9c), however, the system updates the resource allocation more frequently and there is no fixed limit to each of the containers. With this dynamic management scheme, whenever an idling slot is found, Kubernetes can assign jobs without maintaining fairness. Given the same amount of resources, it can reduce the idling time and improve resource utilization, which leads to the reduction of completion time. The similar results are found on Fig. 8d and Fig. 9d, where there is a 100s submission gap between jobs. The gap between jobs introduces more scheduling workload for the system. We still can find the trends that are similar to before. With Docker the resource allocation is stable and Kubernetes manages the running pods with a more dynamic fashion. On one hand, Docker keeps maintaining fair resource allocation, e.g. from 100s to 200s, Job-1 and Job-2 average CPU usages are around 46% each, on the other hand, Kubernetes dynamically adjusts resources to prevent idling, e.g. average CPU usages from 100s to 200s are 51% and 42% for Job-1 and Job-2, respectively. The dynamic resource adjustment leads to performance improvement.

Explicit Resource Limits: Then, we re-conduct Type 3 experiments with an explicit limit for each of the container/pod to understand how resource configuration affects the performance. Please note that this value is an upper bound of the resources that can be occupied by a specific container. The similar of usage patterns are indicated on Figure 8c and Figure 10a. However, with a specific limit on each container, the system scheduler Comparing Figure 9c and Figure 10b, a very different resource management behavior is observed. We can see that the resource usage of each container is capped at 20% of the total amount. With limits, the system lost the ability to dynamically adjust resource allocation. Therefore, we discover the performance downgrade when comparing two experiments on the same platform. Table II presents the completion time of Docker and Kubernetes with default setting (D) and specific limits setting (L). Comparing Docker (D) with Docker (L), the completion time increases around 1% to 6%. The reason lies in the fact that, with specific limit values on each container, it introduces more scheduling overhead for the system to keep tracking the resource consumption. When considering Kubernetes, not only the scheduling overhead is brought by maintaining the limits, but also the explicit limits disable the flexibility for scheduling to assign tasks to maximize resource utilization and preventing idling tasks. As we can see that Job-5 records completion time increase of 38.5% and on average, it increases from 863.6s to 1128.2s, 30.6%.

Table II: Experiments with Specific Limits (Seconds)

Platform	Job-1	Job-2	Job-3	Job-4	Job-5
Docker (D)	1209.1	1232.5	1233.7	1216.4	1229.1
Docker (L)	1277.6	1270.5	1255.3	1278.2	1290.3
Kubernetes (D)	871.3	872.1	840.8	871.9	867.2
Kubernetes (L)	1124.7	1056.6	1125.7	1138.5	1198.4

Parallelism with Multithreading Finally, we study how multithreading configuration can affect the performance. Generally, developers take advantage of multithreading to perform several tasks simultaneously and maximize multi-core hardware. However, incorrect usage of multithreading may result in increased hardware usage cycles and can drastically reduce the system performance.

In these experiments, we use Pytorch as our computing framework. In Pytorch, developers can use a method `torch.set_num_threads(int)` to set the number of threads used for intraop parallelism. The default value is 8, which means that each Pytorch task can launch 8 threads for parallel processing.

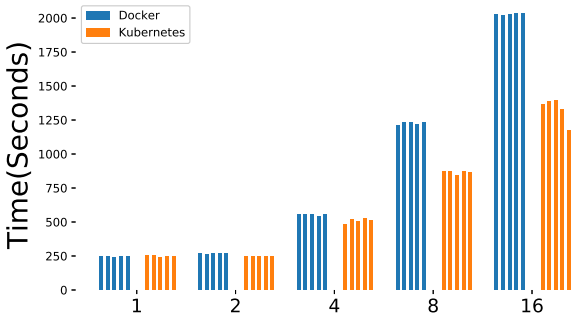
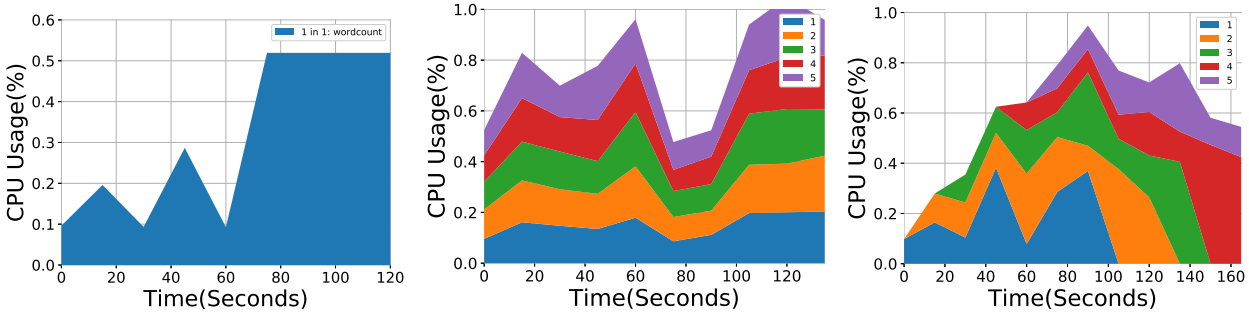


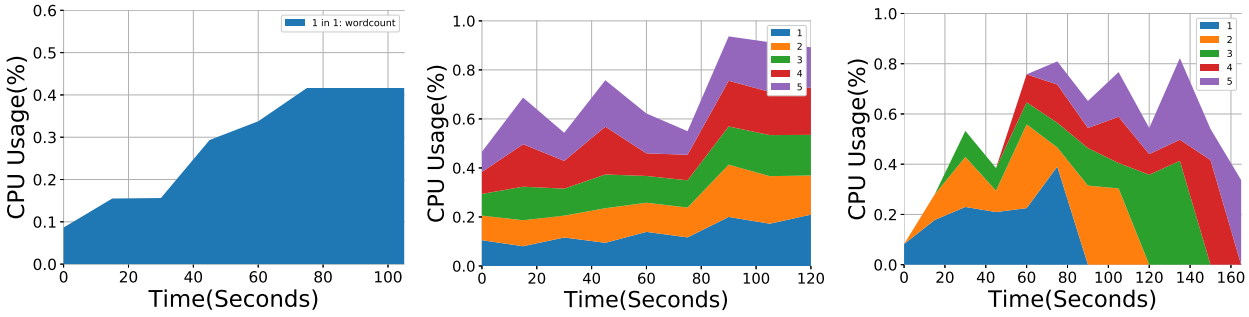
Figure 11: Completion time with various thread numbers

Fig. 11 illustrates the completion time under different thread settings. Obviously, the performance varies a lot. With the default setting 8 threads, the average completion time is 1207.1s and 815.8s for Docker and Kubernetes, respectively. When increasing the value to 16, the average becomes 2025.8s and 1329.4s, which increases 67.8% and 62.9%. The significant performance downgrade is due to the increased overhead for switching between threads. When reducing the number to



(a) 1 in 1 WordCount (b) 5 in 5 WordCount (c) 5 in 5 WordCount with 100s interval

Figure 12: CPU usage for Hadoop Yarn (WordCount) with Docker Platform



(a) 1 in 1 WordCount (b) 5 in 5 WordCount (c) 5 in 5 WordCount with 100s interval

Figure 13: CPU usage for Hadoop Yarn (WordCount) with Kubernetes Platform

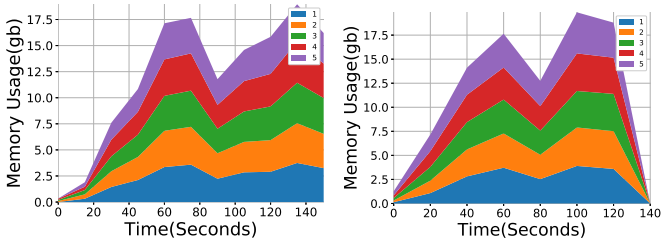
4, the values reduce to 554.2s and 508.6s for Docker and Kubernetes. The average CPU usage for all three settings is at around 97% in average. Given the same resource utilization, a smaller number of threads results in less swapping overhead and lead to better performance. Further reducing the thread number to 1 and 2, they achieve similar completion time 249.0s, 253.9s (Docker) and 248.4s, 249.6s (Kubernetes). Both of them utilize fewer resources than the total availability. This implies that the performance in these experiments mainly depends on the algorithm implementation, not the resources.

Please note that optimal value that can achieve the best performance varies in different systems and workloads. It mainly depends on the details of hardware type, configuration, model characteristics, and algorithm implementation.

2) Big Data Processing: In this subsection, we evaluate the systems with Hadoop Yarn and Spark.

With WordCount (1 Map 1 Reduce) as the representative workload, we conduct three types of experiments 1 in 1, 5 in 5 and 5 in 5 with 20s submission interval. Figure 12 and Figure 13 present the results of Hadoop Yarn. When there is 1 job running in the system, the resource contention is minimum. However, we can see from Figure 12a and Figure 13a, the container/pod fails to consume all the CPU resource (16 cores). This indicates that the WordCount job is not as computation-intensive as deep learning applications. The resource usage between two platforms is similar except the pod in Kubernetes gets more dynamic allocation.

Figure 12b and Figure 13b show the results when 5 WordCount jobs in 5 individual containers/pods. With the figures,



(a) Docker (b) Kubernetes

Figure 14: Memory usage of Hadoop Yarn with 5 in 5 (Type 3) WordCount with specific limits

we can clearly see the Map-Reduce transition phase, where the CPU usage decreases first as the map is finishing and then increase again as the reduction is running.

When submitting a workload with a gap, more scheduling tasks need to be done by the system. Figure 12c and Figure 13c plot the results. Kubernetes can dynamically change resource allocation with respect to new workloads.

Besides CPU usage, Figure 14a and Figure 14b illustrate the memory usage of 5 in 5 experiments. The trend of memory usage is very similar between the two platforms. However, the memory on Docker is released at time 530.6s and with Kubernetes, the memory is returned to the system at 680.8s, which is 28.3% longer than Docker.

Figure 15 illustrate the completion time of the WordCount jobs. Unlike a huge performance difference with deep learning jobs, the completion time of WordCount jobs is within a

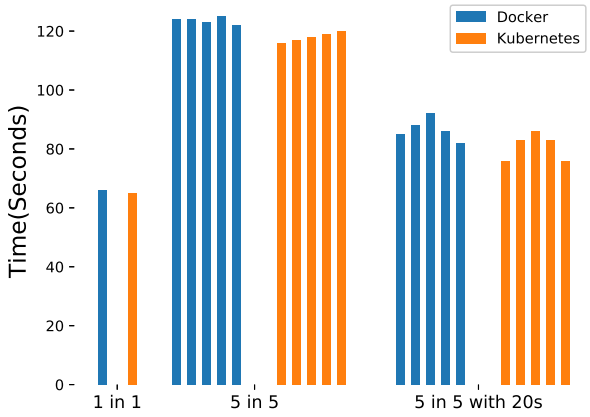


Figure 15: Completion time of WordCount jobs

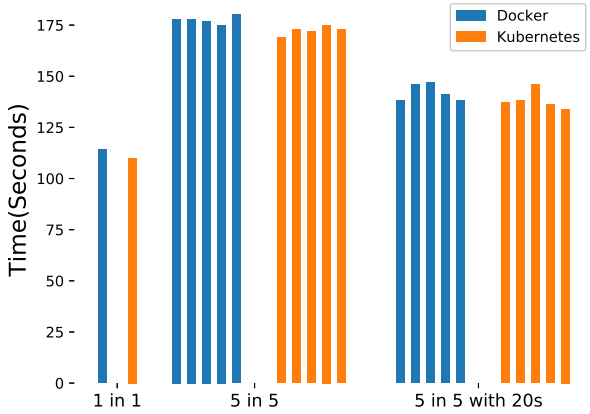


Figure 16: Completion time of PageRank jobs

small range. For example, the average completion time for 1 in 1, 5 in 5 and 5 in 5 with 100s submission interval is 66s v.s. 65s, 123.6s v.s. 118s, and 86.6s v.s. 80.6s. The reason is that WordCount jobs are far less CPU-sensitive than VAE. Additionally, MapReduce based WordCount jobs require diverse types of resources, memory for the data, and network for shuffling.

Figure 17 and Figure 18 present the results for PageRank jobs on Spark with 1 in 1, 5 in 5 and 5 in 5 with 100s submission interval. We find multiple stages from the resource usage patterns, which is a feature of PageRank jobs. In general, jobs in Kubernetes consumes fewer resources since the system reserves part of the resources for itself. Figure 16 plots the completion time of PageRank jobs. The same as WordCount jobs, we find that both Docker and Kubernetes achieve a similar completion time in the experiments.

3) Cluster Environment: In this subsection, we evaluate the two platforms with a 4-node cluster, 1 manager, and 4 workers, which builds with previously mentioned M510 machines. In this cluster, Worker-1 serves as both manager and worker.

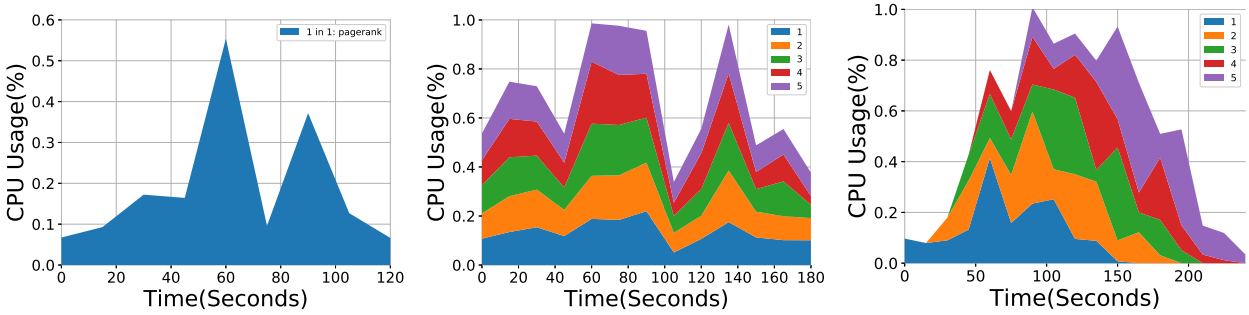
For the experiments, we first randomly select 40 jobs from deep learning applications in Table I. Then, for each job, we randomly generate a submission time with the interval [0, 1000s]. With this setting, we produce more tasks for the system scheduler, which is responsible for various tasks, such as container placement and resource allocation.

Figure 19 plots the results of completion time for each job in the cluster. Since deep learning applications are As shown in the figure, Kubernetes outperforms Docker Swarm in 34 out of 40 jobs. The average completion time is reduced from 2328.5s to 1576.6s, 32.3%. The largest gain is achieved on Job-8, which reduces 60.3%, from 3605.7s to 1430.3s. As expected, the improvement is due to dynamic resource adjustment to prevent idling tasks and increase resource utilization. Fig. 20 and Fig. 21 prove the conclusion since a similar trend is found for both Docker Swarm and Kubernetes as we do in the single node system.

However, Docker Swarm wins significantly on 6 out of 40 jobs. For example, the completion time of Job-22 is 2532.2s for Kubernetes and 1287.1s with Docker Swarm, which is a 96.7% increase. This is due to the fact that, in a cluster environment, the system scheduler not only manages resources on each individual workers, but also decides the workload on a specific worker through initial container placement. With a deeper investigation of the system, we find that the container distribution on Docker Swarm and Kubernetes is different. Fig. 22 and Fig. 23 illustrate the container placement in this experiment. First of all, we discover that for both platforms, Worker-1 hosts less workload than other nodes in the cluster. This is because, in our setting, Worker-1 serves as both Manager that is responsible for system-wide scheduling tasks and Worker, which provides the computing resources for the workloads. Therefore, different platforms have various algorithms to prevent overwhelming the manager. As we can see that Kubernetes distribute the first two jobs (submitted at 4s and 15s) to Worker-1 since, at the very beginning, the manager only involves very limited tasks and system trade it as a normal worker. When more tasks join the cluster, the number of scheduling tasks that the manager handles increases. Therefore, the workload on the manager maintains at the same level until 248s, when Job-10 is submitted. With Docker Swarm, however, it tries to protect its manager from the very beginning. The first container that assigns to Worker-1 is Job-12 at 302s. Consequently, the Worker-1 is idling from 0 to 301s, which leads to significant performance differences.

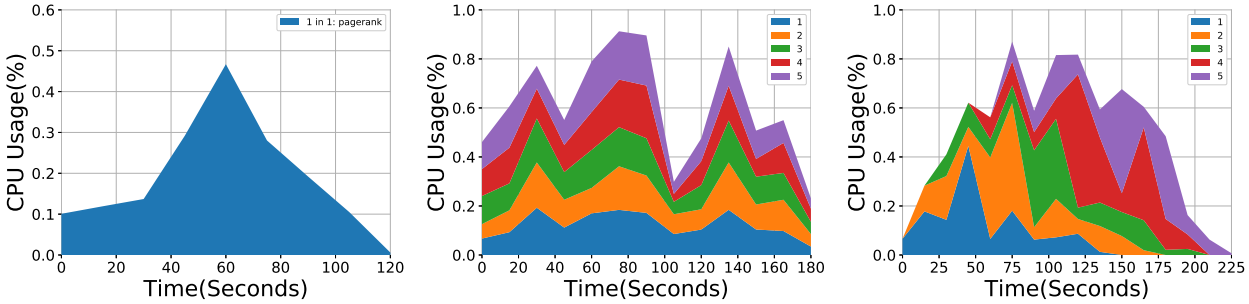
V. Conclusion

In this project, we explored the system performance of two cloud native platforms, Docker and Kubernetes. Together with Prometheus and Grafana, we built a container monitor system to keep tracking the resource usage of each job on the worker nodes. We conducted intensive experiments based on a testbed hosted by the datacenter of NSF Cloudlab. Focusing on two popular computing workloads, deep learning (Tensorflow and Pytorch) and big data (Apache Yarn and Spark) applications, our experiments utilize burst, fixed, and random submission schedules on both single node and cluster environment. The



(a) 1 in 1 PageRank (b) 5 in 5 PageRank (c) 5 in 5 PageRank with 100s interval

Figure 17: CPU usage for Spark (PageRank) with Docker



(a) 1 in 1 PageRank (b) 5 in 5 PageRank (c) 5 in 5 PageRank with 100s interval

Figure 18: CPU usage for Spark (PageRank) with Kubernetes

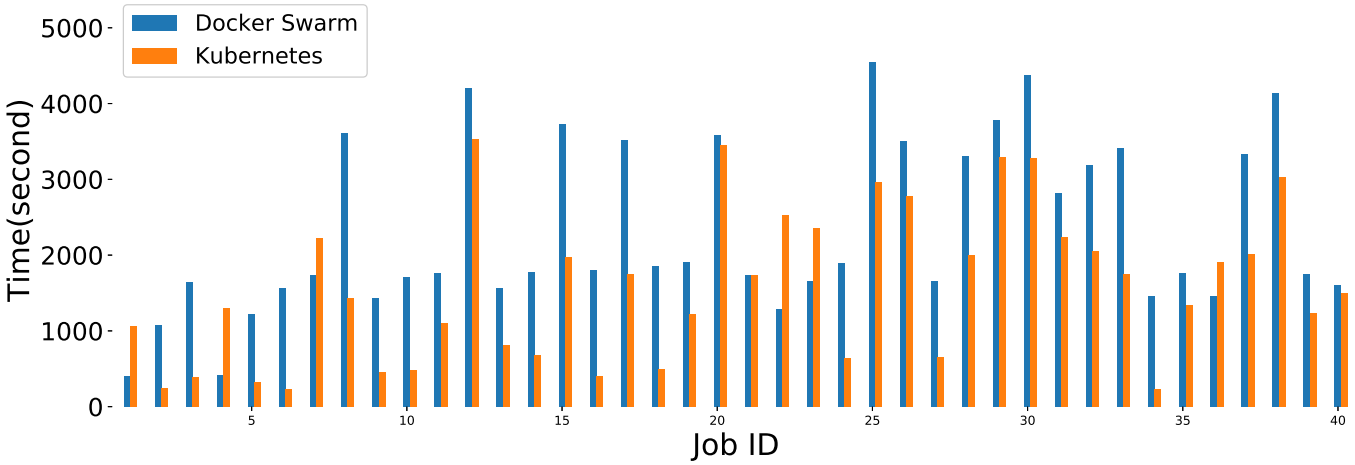


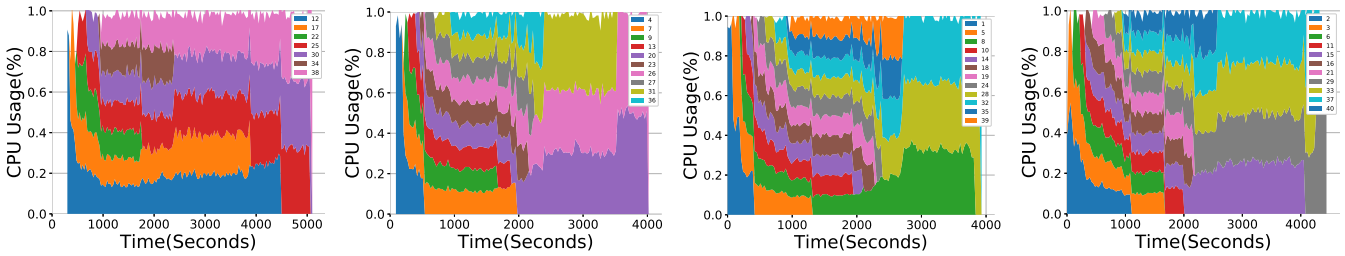
Figure 19: Completion time of 40 deep learning jobs in a 4-node cluster

results demonstrate that considering each individual platform, the completion time was reduced for up to 79.4% and 69.4% by changing the default configurations on Docker and Kubernetes. Comparing two platforms, we discover that Docker platform delayed resource release process for up to 116.7% for short-lived deep learning jobs. Furthermore, the completion time increased up to 96.7% in the cluster environment due to different resource management schemes.

As future work, we will consider designing algorithms to dynamically adjust the thread numbers for Tensorflow and Pytorch to boost system performance by increase resource utilization. In addition, we plan to explore the system performance from the perspective of container recovery and migration to unfold an optimized container management scheme. Finally, we intend to conduct experiments to understand the system performance of the two platforms with long-lived workloads, e.g. web and database services.

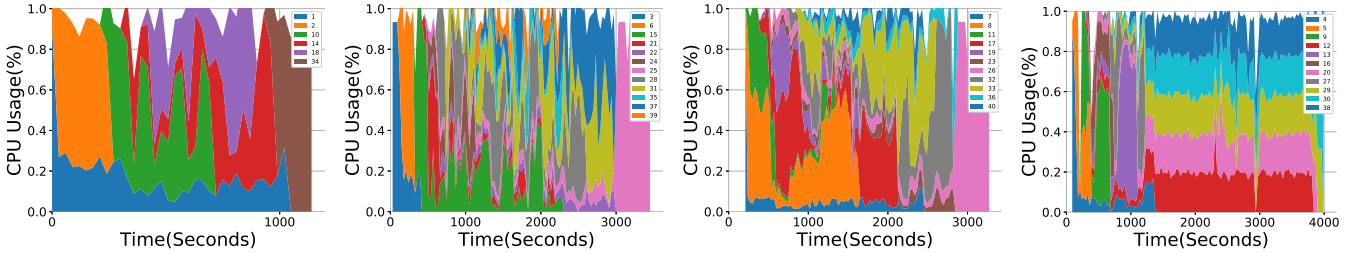
REFERENCES

[1] "Forbes report," https://www.forbes.com/sites/louiscolumbus/2015/01/24/roundup-of-cloud-computing-forecasts-and-market-estimates-2015.
[2] "Amazon web service," https://aws.amazon.com/.
[3] "Microsoft azure," https://azure.microsoft.com/en-us/.
[4] "Google cloud platform," https://cloud.google.com/.
[5] "Docker," https://www.docker.com/.



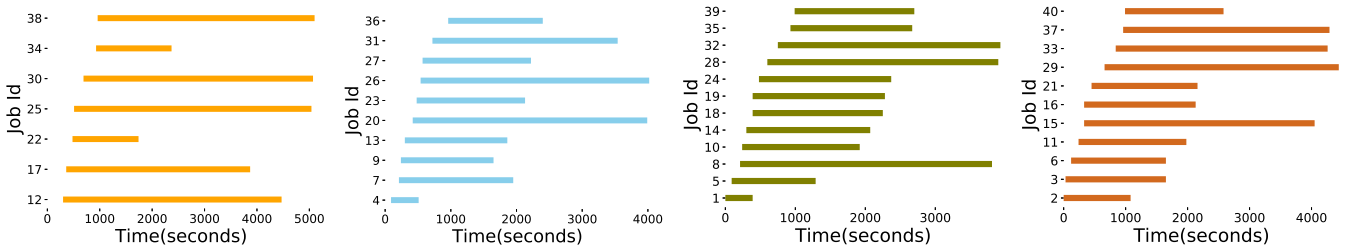
(a) Worker-1 (b) Worker-2 (c) Worker-3 (d) Worker-4

Figure 20: Container Placement with Docker Swarm



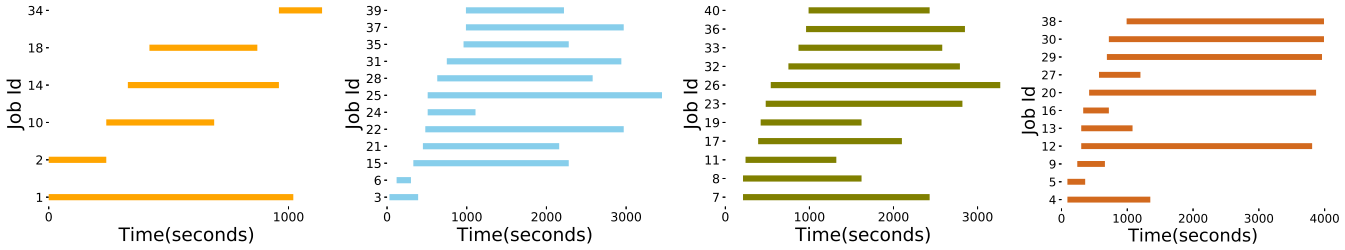
(a) Worker-1 (b) Worker-2 (c) Worker-3 (d) Worker-4

Figure 21: Container Placement with Kubernetes



(a) Worker-1 (b) Worker-2 (c) Worker-3 (d) Worker-4

Figure 22: Container Placement with Docker Swarm



(a) Worker-1 (b) Worker-2 (c) Worker-3 (d) Worker-4

Figure 23: Container Placement with Kubernetes

[6] "Docker swarm," https://docs.docker.com/engine/swarm/.

[7] "kubernetes," https://kubernetes.io/.

[8] "Aws history," https://mediatemple.net/blog/news/brief-history-aws/.

[9] "Facebook," https://facebook.com/.

[10] "Twitter," https://twitter.com/.

[11] "Google gmail," https://gmail.com.

[12] "Outlook," https://outlook.com/.

[13] X. Chen, L. Cheng, C. Liu, Q. Liu, J. Liu, Y. Mao, and J. Murphy, "A woa-based optimization approach for task scheduling in cloud computing systems," *IEEE Systems Journal*, 2020.

[14] Y. Mao, J. Oak, A. Pompili, D. Beer, T. Han, and P. Hu, "Draps: Dynamic and resource-aware placement scheme for docker containers in a heterogeneous cluster," in *2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC)*. IEEE, 2017, pp. 1–8.

[15] Y. Mao, J. Wang, J. P. Cohen, and B. Sheng, "Pasa: Passive broadcast for smartphone ad-hoc networks," in *2014 23rd International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 2014, pp. 1–8.

[16] Y. Mao, V. Green, J. Wang, H. Xiong, and Z. Guo, "Dress: Dynamic resource-reservation scheme for congested data-intensive computing platforms," in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*. IEEE, 2018, pp. 694–701.

[17] A. Acharya, Y. Hou, Y. Mao, M. Xian, and J. Yuan, "Workload-aware

task placement in edge-assisted human re-identification," in *2019 16th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. IEEE, 2019, pp. 1–9.
[18] A. Acharya, Y. Hou, Y. Mao, and J. Yuan, "Edge-assisted image processing with joint optimization of responding and placement strategy," in *2019 International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. IEEE, 2019, pp. 1241–1248.
[19] H. H. Harvey, Y. Mao, Y. Hou, and B. Sheng, "Edos: Edge assisted offloading system for mobile devices," in *2017 26th International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 2017, pp. 1–9.
[20] F. Xu, F. Liu, H. Jin, and A. V. Vasilakos, "Managing performance overhead of virtual machines in cloud computing: A survey, state of the art, and future directions," *Proceedings of the IEEE*, vol. 102, no. 1, pp. 11–31, 2013.
[21] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and linux containers," in *2015 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2015, pp. 171–172.
[22] P. Sharma, L. Chaufournier, P. Shenoy, and Y. Tay, "Containers and virtual machines at scale: A comparative study," in *Proceedings of the 17th International Middleware Conference*. ACM, 2016, p. 1.
[23] S. F. Piraghaj, A. V. Dastjerdi, R. N. Calheiros, and R. Buyya, "Containercloudsim: An environment for modeling and simulation of containers in cloud data centers," *Software: Practice and Experience*, vol. 47, no. 4, pp. 505–521, 2017.
[24] Y. Tay, K. Gaurav, and P. Karkun, "A performance comparison of containers and virtual machines in workload migration context," in *2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW)*. IEEE, 2017, pp. 61–66.
[25] J. Bhimani, Z. Yang, M. Leeser, and N. Mi, "Accelerating big data applications using lightweight virtualization framework on enterprise cloud," in *2017 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2017, pp. 1–7.
[26] Q. Zhang, L. Liu, C. Pu, Q. Dou, L. Wu, and W. Zhou, "A comparative study of containers and virtual machines in big data environment," *arXiv preprint arXiv:1807.01842*, 2018.
[27] A. Khan, "Key characteristics of a container orchestration platform to enable a modern application," *IEEE Cloud Computing*, vol. 4, no. 5, pp. 42–48, 2017.
[28] C. Xu, K. Rajamani, and W. Felter, "Nbwguard: Realizing network qos for kubernetes," in *Proceedings of the 19th International Middleware Conference Industry*. ACM, 2018, pp. 32–38.
[29] Y. Fu, S. Zhang, J. Terrero, Y. Mao, G. Liu, S. Li, and D. Tao, "Progress-based container scheduling for short-lived applications in a kubernetes cluster," in *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 2019, pp. 278–287.
[30] W. Zheng, Y. Song, Z. Guo, Y. Cui, S. Gu, Y. Mao, and L. Cheng, "Target-based resource allocation for deep learning applications in a multi-tenancy system," in *2019 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2019, pp. 1–7.
[31] W. Zheng, M. Tynes, H. Gorelick, Y. Mao, L. Cheng, and Y. Hou, "Flowcon: Elastic flow configuration for containerized deep learning applications," in *Proceedings of the 48th International Conference on Parallel Processing*, 2019, pp. 1–10.
[32] "Tensorflow," https://www.tensorflow.org/.
[33] "Pytorch," https://pytorch.org/.
[34] "Apache spark," https://spark.apache.org/.
[35] "Cloud native computing foundation," https://www.cncf.io/.
[36] "Docker hub," https://hub.docker.com/.
[37] "Yaml," https://yaml.org/.
[38] "Prometheus," https://prometheus.io.
[39] "Grafana," https://grafana.com.
[40] "Hadoop yarn," https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html.
[41] "Nsf cloudlab," https://cloudlab.us/.